同济大学

毕业设计 (论文) 开题报告

课题名称	面向代码智能体的开发任务上下文追踪与代码变更溯源系统设计与实现
副 标 题	
学　　院	计算机科学与技术学院
专　　业	计算机科学与技术
学生姓名	待填写
学　　号	待填写

_____ 年 ____ 月 ____ 日

摘要

开发者借助代码智能体完成代码修改、执行与测试已成为常态，但对话、变更、工具调用与测试结果分散在不同界面，缺少以开发任务为中心的组织，使“为何这样修改、依据什么验证、中断后如何接续”难以回答。本课题面向单个开发者，研究任务、对话、变更、工具调用与测试之间的关联与组织方法，设计并实现支持任务上下文追踪与变更溯源的 Web 原型，并从概念与模型、采集与关联、溯源与续接、评估四个方面提出初步实施方案。本研究定位为小样本探索性预研，结论以 6—8 个任务与 4—5 名受试者的描述统计结果表述。

关键词：代码智能体；任务上下文；变更溯源；中断恢复；证据链

一、毕业设计（论文）课题背景

1．课题来源及研究的目的和意义

近年来，代码智能体（code agent）逐渐应用于日常软件开发。开发者以自然语言描述任务，由智能体完成代码分析、文件修改、工具调用、测试执行与问题排查。与代码补全不同，智能体的产出并非一段建议代码，而是一组带有工具调用与执行反馈的操作序列；开发过程由此从“人写代码”转变为“人下指令、智能体实施、人审查验证”的协作过程。这一转变在提高任务吞吐的同时，也使“过程是否可理解、可核查、可接续”成为开发者必须面对的问题。

然而，现有工具通常将过程信息分散于不同界面或不同类型的记录之中：对话留存于会话窗口，代码差异留存于 diff 视图，工具调用留存于终端日志，测试结果留存于测试报告，版本演化留存于 Git 历史。这些记录之间既缺少共同标识，也缺少以开发任务为中心的组织。由此产生三方面问题：一是任务视角缺失，在多轮修改、多任务并行与中断重启后难以掌握当前进展；二是溯源断裂，代码因何而改、依据哪次测试验证，只能依靠人工查找与回忆；三是上下文丢失，重新开始工作时需重读长对话与大量差异。

本课题面向单个开发者使用代码智能体完成开发任务的场景，研究开发任务、对话、代码变更、工具调用与测试结果之间的关联与组织方法，设计并实现支持任务上下文追踪与变更溯源的 Web 原型，使开发者能够围绕任务、文件或代码片段审视“为何修改、依据何在、验证结果如何”，并在任务中断后恢复必要的上下文。

本课题的意义体现在三个层面。理论层面，将“任务上下文”与“变更溯源”的概念从 IDE 与桌面场景迁移至智能体驱动的“指令—工具调用—变更—测试”序列，建立其与 PROV 的映射，并讨论任务边界判定、异构记录关联与证据充分性等表示问题。实践层面，将关联、溯源与恢复实现为可查询、可复现的原型，为单个开发者提供可操作的证据链，而非停留于日志浏览。教学层面，完整经历“问题识别—文献定位—模型设计—原型实现—实证评估”的研究流程，并明确说明小样本评估结论的适用边界。

2．文献综述

本节按“代码智能体与软件工程自动化—变更分析与代码历史—任务上下文与中断恢复—过程溯源与证据链—工程工具现状”五条脉络梳理相关研究，并在此基础上进行述评。文献检索以 Google Scholar、DBLP、arXiv、ACM Digital Library 与《软件学报》为主，检索词包括 code agent、software engineering agent、task context、provenance、program comprehension、resumption 等。

（一）代码智能体与软件工程自动化

SWE-bench 以真实 GitHub issue 检验端到端修复，该文报告当时最优模型仅解决 1.96% 的任务 [1]；针对代码生成能力的系统评测则提供了更一般的基准结果 [2]。此后，SWE-agent[3]、OpenHands[4] 与 CodeAct[5] 等以工具调用与执行反馈为核心的方法明显提升了该指标；MetaGPT[6] 与 ChatDev[7] 探索多智能体协作，亦有研究主张以更简化的流程取得可比效果 [8]。相关综述从软件工程全流程 [9-10]、检索增强 [11] 与测试生成 [12] 等角度总结了研究进展。

能力提升并未消除使用成本。一项对照实验报告称，Copilot 使任务完成速度提升约 55.8%[13]，但开发者仍需进行额外审查与验证 [14-15]，且在采纳生成代码前通常需先理解并验证其行为 [16-17]。更为重要的是，现有评价多以最终修复率或生成质量为指标，过程证据——哪条指令触发修改、哪次工具调用实施修改、哪次测试完成验证——并不在评价范围之内。这表明：智能体的产出若要被开发者信任并接续，必须伴随可核查的过程证据。

（二）变更分析与代码历史

软件仓库挖掘为变更与问题之间的关联提供了方法基础：SZZ 依据修复提交所引用的缺陷报告回溯引入缺陷的变更 [18]，Kagdi 等综述了仓库挖掘的任务

与方法 [19]，Robbes 与 Lanza 主张以变更为中心分析软件演化 [20]；GumTree 通过语法树差分生成比行级 diff 更精确的变更脚本 [21]，CodeShovel 重建方法级代码历史 [22]。上述方法回答了“改了什么”，其关联对象是提交与缺陷报告，而非“由哪条指令触发、由哪次测试验证”。

（三）任务上下文与中断恢复

以任务为单位组织开发上下文并非新的研究命题。Mylyn 提出任务上下文模型，通过任务聚焦界面按当前任务过滤 IDE 代码元素 [23]；TaskTracer 在桌面环境中将文件、邮件与网页归集到任务 [24]；ActivitySpace 跨应用推荐相关代码元素 [25]；Codebook 建立代码实体与提交、缺陷、邮件之间的关联图 [26]。在中断恢复方面，Parnin 与 Rugaber 归纳了恢复策略对线索丰富度的依赖 [27]，Parnin 与 DeLine 通过受控实验比较了情景线索与语义线索 [28]，Safer 与 Murphy 研究了任务边界识别界面 [29]，Robbes 与 Lanza 刻画了开发会话的结构 [30]。上述工作构成本课题的基础，但其前提是开发者直接在 IDE 或桌面环境中操作、任务由开发者显式声明，线索主要来自编辑与窗口事件；在智能体事件流中，任务边界通常未被显式声明，线索形态也由文本、工具调用与执行结果构成。

（四）过程溯源与证据链

数据库领域提出了溯源的经典框架 [31]；PROV-O 将溯源抽象为实体、活动与代理及其派生关系，已成为跨系统交换标准 [32]；PROV-AGENT 覆盖智能体交互中的提示、响应、决策与工具调用 [33]。Whyline 通过“为什么”式提问组织执行证据 [34]，noWorkflow 将脚本执行过程捕获为可查询的溯源图 [35]。上述工作证明了证据链的价值，但尚未面向“指令—工具调用—变更—测试”这一异构序列建模，也未将验证依据纳入证据图。

（五）工程工具现状与研究述评

工程实践中已有工具亦可归入上述方向：智能体客户端（Claude Code[36]、Cursor[37]、Cline[38]）以会话或文件为单位提供历史记录与检查点，但检查点随会话结束或保留期届满而失效；LLM 可观测性平台（Langfuse[39]）与 OpenTelemetry 生成式 AI 语义约定 [40] 记录调用链与耗时，未与代码变更、测试结果进行语义对齐。

上述工作可归为四个方向：智能体能力、变更分析、任务上下文与中断恢复、

过程溯源。四者均有所积累，但尚未同时满足以下三点：（1）以任务为根节点统一关联对话、变更、工具调用与测试；（2）变更粒度达到 hunk 并支持双向查询；（3）在证据绑定约束下利用大模型生成摘要与进展整理。需要说明的是，“以任务为单位组织上下文”本身并非研究空白，Mylyn[23] 与 Parnin 等 [27-28] 已提供模型与实证基础；本课题的差异在于研究对象从“人在 IDE 中的编辑事件”转变为“智能体驱动的指令—工具调用—变更—测试序列”，并将验证依据纳入证据链。在所及文献范围内，尚未见到同时满足上述三点的系统；文献综述定稿阶段将补充系统检索，并记录检索式与命中数。

二、毕业设计（论文）方案介绍

1．研究问题与研究内容

本课题关注的核心问题是：单个开发者使用代码智能体完成开发任务时，如何将分散于对话、工具调用、代码变更与测试记录中的过程信息组织起来，使之可追溯、可核查，并在任务中断后支持上下文恢复。该问题包含三个相互关联的方面：一是采用何种信息模型统一表示异构记录，使任务、对话、变更、执行与测试之间能够建立稳定关联；二是依据哪些线索将上述记录归并到具体任务，并判定任务的起止边界；三是如何围绕代码变更组织证据、呈现任务进展，并在中断后恢复必要的上下文。三者分别对应表示、关联与恢复三个层面，评估则贯穿其中。研究方法上，拟采用文献分析、模型设计、原型实现与实证评估相结合的方式，先建立表示与关联方法，再通过原型检验其可用性。

围绕上述问题，本课题的研究内容主要包括五个方面。一是任务上下文的信息模型与统一事件表示，抽象开发任务、对话轮次、消息、工具调用、文件变更、测试运行、测试用例结果与检查点等实体及其关系，设计可扩展的事件模式，使不同来源的记录进入同一表示框架。二是上下文采集与关联，接入代码智能体的会话与工具调用、文件系统与 Git、测试报告，将编辑动作归一化为变更块，并利用时间邻近、显式引用与文件级位置映射等线索建立变更与对话之间的多对多关联。三是变更溯源与证据链，以变更块为单位记录触发来源、实施过程、修改前后快照、验证依据与后续演化，支持从代码到原因、从原因到代码的双向查询。四是交互界面与任务续接，设计任务中心、任务时间线与变更证据侧栏等视图，并在任务中断后生成任务快照，呈现已完成、未完成与待验证事项。五是大

语言模型辅助与评估，在证据绑定约束下生成任务摘要与进展整理，并通过典型编程任务考察系统在代码理解、修改原因分析与任务衔接方面的作用。

2．初步实施方案

本课题的初步实施方案按“概念与模型—采集与关联—溯源与续接—评估”四个模块组织。四个模块对应研究推进的不同环节：概念与模型回答“表示什么”，采集与关联回答“如何获得并建立联系”，溯源与续接回答“如何查询与使用”，评估回答“方案是否有效”。四者并非软件工程的层次划分，而是相互约束的研究环节，其关系如图 2-1 所示。

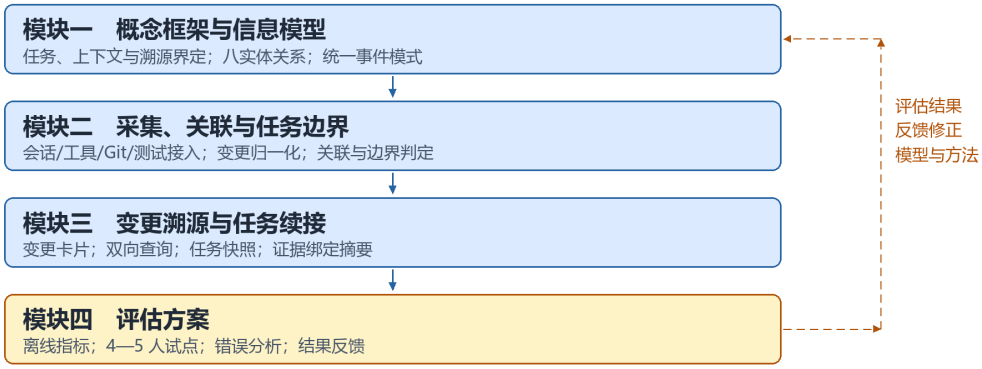


图 2-1 研究技术路线

（一）概念框架与信息模型

本模块界定开发任务、任务上下文与变更溯源三个核心概念，并提出可计算的信息模型。开发任务指开发者发起的一次有明确目标的开发工作单元；任务上下文指与该任务相关的对话、变更、执行、测试、决策与未完成事项的集合；变更溯源指从一段代码变更出发，追溯其触发来源、实施过程、验证依据与后续演化的证据链。在此基础上，将任务上下文刻画为以任务为根的有向异构图：节点包括任务、对话轮次、消息、工具调用、文件变更、测试运行、测试用例结果与检查点，边包括包含、触发、产出、验证、演化与快照等关系，其基本结构如图 2-2 所示。以任务为根而非以会话或文件为根，是为了让跨会话、跨文件的记录围绕同一目标聚合，并为后续的边界判定与证据查询提供统一入口。该模型与 PROV-O[32] 的基本思想一致，但针对智能体事件流增加了任务边界、变更块与验证依据等约束。事件采用统一的 JSON 结构表示，字段涵盖标识、类型、时间、主体、来源、载荷与关联；模型需满足变更标识稳定、消息与变更多对多、测试

结果双向指向等基本约束。上述概念与模型为后续采集、关联与查询提供共同语言，其具体表结构与索引方式将在原型实现阶段细化。

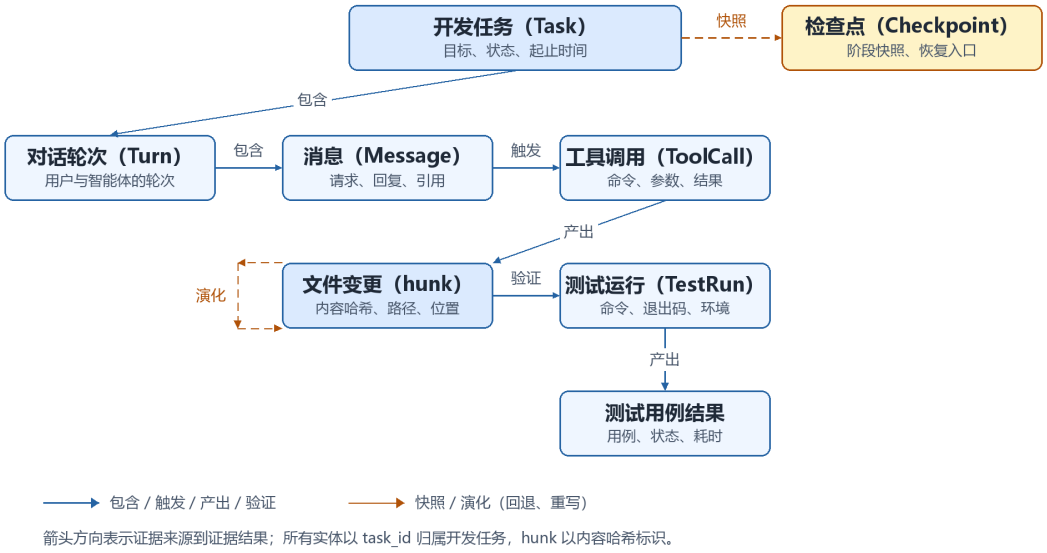


图 2-2 任务上下文的信息模型

（二）上下文采集、关联与任务边界

本模块研究如何从真实开发过程中获得上述实体，并建立它们之间的关联。采集对象包括代码智能体的会话与工具调用、文件系统与 Git 变更、测试运行与结果；接入方式以 Claude Code 的官方钩子与 stream-json 为主，本地日志与 Git 作为补充。由于不同工具的编辑动作形式不一，需要将“旧串 → 新串”、补丁、整文件写入等统一归一化为变更块；语法感知的符号定位可借助 tree-sitter[41] 一类解析器完成。关联层面，拟综合利用时间邻近、显式引用（路径、符号、行号、命令、测试名）与文件级位置映射三类线索，为每条消息与每个变更块计算关联强度。由于同一变更可能由多轮消息共同促成，关联结果保留置信度与多对多关系，而不强行归并为单一来源。任务边界判定则依据空闲时长、显式指令、目标切换与提交动作等特征，采用可解释的规则打分，以便向开发者说明判定依据。该模块的重点不在于追求单点最优，而在于形成一套可复现、可解释的关联流程，其参数与阈值将在小规模验证集上调整并报告敏感性。

（三）变更溯源与任务续接

本模块将前两个模块的表示与关联转化为开发者可查询、可使用的证据链。

对每个变更块生成变更卡片，记录触发消息、相关工具调用、修改前后快照、验证测试与后续演化；查询支持从代码到原因、从原因到代码两个方向。任务续接以任务快照为核心：快照由任务上下文图生成，将事项分为已完成、未完成与待验证三类，并附带指向证据的链接，使开发者能够从摘要直接跳转到对应的消息、变更或测试结果。原型拟采用 Web 形态，提供任务中心、任务时间线与变更证据侧栏三个视图；文件与代码片段视角作为时间线的过滤与详情，不单独作为第四视图。三个视图分别回答“当前有哪些任务”“任务经过了什么”“某处变更依据何在”，共同构成从任务到证据的浏览路径。大语言模型用于生成任务摘要与进展整理，但输出须绑定证据标识，证据不足时弃答，以降低无依据结论的风险。

（四）评估方案

本模块检验前三个模块的构想是否可行。研究定位为小样本探索性预研，不追求确证性结论。离线评测拟在 2 个中小规模开源仓库的 6—8 个任务上进行，任务覆盖缺陷修复、功能新增与代码重构，其中部分可改写自 SWE-bench 与 SWE-rebench[42] 的真实 issue；考察关联准确率、任务边界判定 F1、溯源查询正确率与查询响应时间等指标。用户研究拟招募 4—5 名具备编程经验的同专业学生，在自我中断情境下比较“Git 日志 + 原始会话记录”与“完整系统”两种条件，观察恢复用时、理解要点命中数与问题定位用时，并以 SUS[43] 作为可用性指标，NASA-TLX[44] 作为可选的认知负荷观察。受样本量限制，结果以描述统计与逐人差值呈现，不设显著性结论；同时通过任务构造与评测分离、第三方招募、数据脱敏等方式控制实验者偏差与伦理风险。离线指标用于检验方法的客观正确性，用户研究用于观察其在真实恢复情境中的可用性，二者互为补充。

四个模块的推进顺序为：先完成概念与模型、采集与关联的方法设计，再实现溯源与续接原型，最后开展评估，并将评估结果反馈至模型与方法。各模块的阶段性产出作为下一模块的输入，具体任务与周次见下文进度安排。

3．工作进度安排

本课题按任务书的教学周安排推进，六个阶段如表 2-1 与图 2-3 所示。第 9 周进行开题答辩，开题材料在第 8 周前完成；中期检查与答辩按学院统一安排。时间安排以教学周计，具体以学院通知为准。

表 2-1 研究进度安排

阶段	教学周	主要工作	阶段成果
1 文献研究与产品调研	1—8	文献综述与竞品分析；确定研究问题、评价标准与实验任务；完成开题报告与答辩准备	文献综述、开题报告
2 关键方法设计与实现	9—14	信息模型与事件模式、编辑动作归一化、关联与边界判定、证据链查询	方法设计、实验脚本
3 系统设计与实现	15—20	三个视图、任务快照与续接、大模型摘要与证据校验、原型集成	原型 v0.9
4 系统验证与测试	21—22	6—8 个典型任务实验、4—5 人试点、错误分析与效度检查	实验数据、评估报告
5 毕业设计论文撰写	23—28	实验分析、论文初稿、图表与复现材料	论文初稿
6 论文审阅与修改	29—32	论文修改、查重、定稿与答辩材料	论文定稿、答辩材料

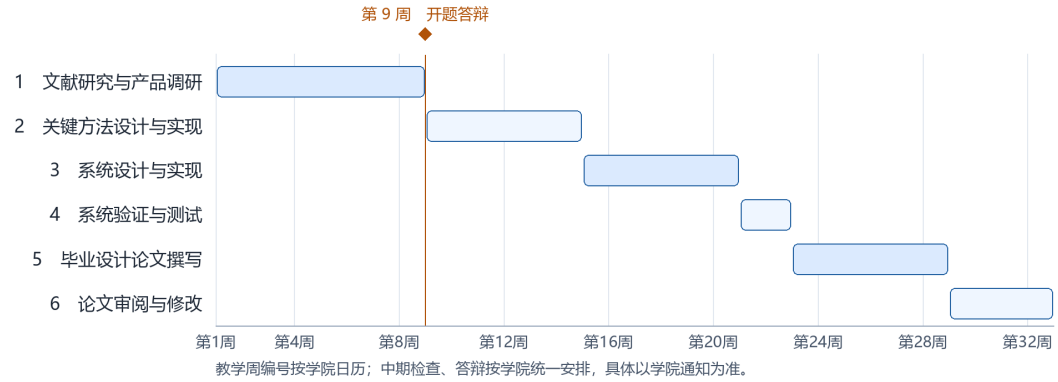


图 2-3 研究进度甘特图

三、毕业设计（论文）的主要参考文献

（说明：会议文献按 GB/T 7714-2015 著录，出版地与出版者从略；网络资源引用日期统一为 2026-10-27。）

[1] JIMENEZ C E, YANG J, WETTIG A, et al. SWE-bench: can language models resolve real-world GitHub issues?[C]//Proceedings of the 12th International Conference on Learning Representations. 2024. arXiv: 2310.06770.

[2] CHEN M, TWOREK J, JUN H, et al. Evaluating large language models trained on code[EB/OL]. (2021-07-07)[2026-10-27]. https://arxiv.org/abs/2107.03374.

[3] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: agent-computer interfaces enable automated software engineering[C]//Advances in Neural Information Processing Systems 37. 2024. arXiv: 2405.15793.

[4] WANG X, LI B, SONG Y, et al. OpenHands: an open platform for AI software developers as generalist agents[C]//Proceedings of the 13th International Conference on Learning Representations. 2025. arXiv: 2407.16741.

[5] WANG X, CHEN Y, YUAN L, et al. Executable code actions elicit better LLM agents[C]//Proceedings of the 41st International Conference on Machine Learning. 2024. arXiv: 2402.01030.

[6] HONG S, ZHUGE M, CHEN J, et al. MetaGPT: meta programming for a multi-agent collaborative framework[C]//Proceedings of the 12th International Conference on Learning Representations. 2024. arXiv: 2308.00352.

[7] QIAN C, LIU W, LIU H, et al. ChatDev: communicative agents for software development[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. 2024: 15174-15186. DOI: 10.18653/v1/2024.acl-long.810.

[8] XIA C S, DENG Y, DUNN S, et al. Agentless: demystifying LLM-based software engineering agents[EB/OL]. (2024-07-01)[2026-10-27]. https://arxiv.org/abs/2407.01489.

[9] HOU X, ZHAO Y, LIU Y, et al. Large language models for software engineering: a systematic literature review[J]. ACM Transactions on Software Engineering and Methodology, 2024, 33(8). DOI: 10.1145/3695988.

[10] FAN A, GOKKAYA B, HARMAN M, et al. Large language models for software engineering: survey and open problems[EB/OL]. (2023-10-05)[2026-10-27]. https://arxiv.org/abs/2310.03533.

[11] 张犬俊, 谢杨, 房春荣, 等. 检索增强生成在软件工程中的应用综述 [J]. 软件学报, 2026, 37(3): 1316-1339. DOI: 10.13328/j.cnki.jos.007567.

[12] 汪莹, 字千成, 彭鑫, 等. 基于大语言模型的故障复现测试用例生成方法 [J]. 软件学报, 2026, 37(4): 1690-1714. DOI: 10.13328/j.cnki.jos.007474.

[13] PENG S, KALLIAMVAKOU E, CIHON P, et al. The impact of AI on developer productivity: evidence from GitHub Copilot[EB/OL]. (2023-02-13)[2026-10-27]. https://arxiv.org/abs/2302.06590.

[14] MOZANNAR H, BANSAL G, FOURNEY A, et al. Reading between the lines: modeling user behavior and costs in AI-assisted programming[C]//Proceedings of the CHI Conference on Human Factors in Computing Systems. 2024: 1-16. DOI: 10.1145/3613904.3641936.

[15] PERRY N, SRIVASTAVA M, KUMAR D, et al. Do users write more insecure code with AI assistants?[C]//Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. 2023: 2785-2799. DOI: 10.1145/3576915.3623157.

[16] BARKE S, JAMES M B, POLIKARPOVA N. Grounded Copilot: how programmers interact with code-generating models[J]. Proceedings of the ACM on Programming Languages, 2023, 7(OOPSLA1): 85-111. DOI: 10.1145/3586030.

[17] VAITHILINGAM P, ZHANG T, SO S. Expectation vs. experience: evaluating the usability of code generation tools powered by large language models[C]//Proceedings of the CHI Conference on Human Factors in Computing Systems Extended Abstracts. 2022: 1-7. DOI: 10.1145/3491101.3519665.

[18] ŚLIWERSKI J, ZIMMERMANN T, ZELLER A. When do changes induce

fixes?[C]//Proceedings of the 2005 International Workshop on Mining Software Repositories. 2005: 1-5. DOI: 10.1145/1083142.1083147.

[19] KAGDI H, COLLARD M L, MALETIC J I. A survey and taxonomy of approaches for mining software repositories in the context of software evolution[J]. Journal of Software Maintenance and Evolution: Research and Practice, 2007, 19(2): 77-131. DOI: 10.1002/smr.344.

[20] ROBBES R, LANZA M. A change-based approach to software evolution[J]. Electronic Notes in Theoretical Computer Science, 2007, 166: 93-109. DOI: 10.1016/j.entcs.2006.06.015.

[21] FALLERI J R, MORANDAT F, BLANC X, et al. Fine-grained and accurate source code differencing[C]//Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering. 2014: 313-324. DOI: 10.1145/2642937.2642982.

[22] GRUND F, CHOWDHURY S A, BRADLEY N C, et al. CodeShovel: constructing method-level source code histories[C]//Proceedings of the 43rd International Conference on Software Engineering. 2021: 1510-1522. DOI: 10.1109/ICSE43902.2021.00135.

[23] KERSTEN M, MURPHY G C. Using task context to improve programmer productivity[C]//Proceedings of the 14th ACM SIGSOFT International Symposium on Foundations of Software Engineering. 2006: 1-11. DOI: 10.1145/1181775.1181777.

[24] DRAGUNOV A N, DIETTERICH T G, JOHNSRUDE K, et al. TaskTracer: a desktop environment to support multi-tasking knowledge workers[C]//Proceedings of the 10th International Conference on Intelligent User Interfaces. 2005: 75-82. DOI: 10.1145/1040830.1040855.

[25] BAO L, YE D, XING Z, et al. ActivitySpace: a remembrance framework to support interapplication information needs[C]//Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering. 2015: 864-869. DOI: 10.1109/ASE.2015.90.

[26] BEGEL A, KHOO Y P, ZIMMERMANN T. Codebook: discovering and exploiting relationships in software repositories[C]//Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering. 2010: 125-134. DOI: 10.1145/1806799.1806821.

[27] PARNIN C, RUGABER S. Resumption strategies for interrupted programming tasks[J]. Software Quality Journal, 2011, 19(1): 5-34. DOI: 10.1007/s11219-010-9104-9.

[28] PARNIN C, DELINE R. Evaluating cues for resuming interrupted programming tasks[C]//Proceedings of the SIGCHI Conference on Human Factors in Computing Systems. 2010: 93-102. DOI: 10.1145/1753326.1753342.

[29] SAFER I, MURPHY G C. Comparing episodic and semantic interfaces for task boundary identification[C]//Proceedings of the 2007 Conference of the Center for Advanced Studies on Collaborative Research. 2007: 229-243. DOI: 10.1145/1321211.1321235.

[30] ROBBES R, LANZA M. Characterizing and understanding development sessions[C]//Proceedings of the 15th IEEE International Conference on Program Comprehension. 2007: 155-166. DOI: 10.1109/ICPC.2007.12.

[31] CHENEY J, CHITICARIU L, TAN W C. Provenance in databases: why, how, and where[J]. Foundations and Trends in Databases, 2009, 1(4): 379-474. DOI: 10.1561/1900000006.

[32] W3C. PROV-O: the PROV ontology[EB/OL]. (2013-04-30)[2026-10-27]. https://www.w3.org/TR/prov-o/.

[33] SOUZA R, GUEROUDJI A, DEWITT S, et al. PROV-AGENT: unified provenance for tracking AI agent interactions in agentic workflows[C]//Proceedings of the 21st IEEE International Conference on e-Science. 2025. arXiv: 2508.02866.

[34] KO A J, MYERS B A. Designing the Whyline: a debugging interface for asking questions about program behavior[C]//Proceedings of the SIGCHI Conference on Human Factors in Computing Systems. 2004: 151-158. DOI: 10.1145/985692.985712.

[35] PIMENTEL J F, MURTA L, BRAGANHOLO V, et al. noWorkflow: a tool for collecting, analyzing, and managing provenance from Python scripts[J]. Proceedings of the VLDB Endowment, 2017, 10(12): 1841-1844. DOI: 10.14778/3137765.3137789.

[36] Anthropic. Claude Code documentation: overview[EB/OL]. [2026-10-27]. https://code.claude.com/docs/en/overview.

[37] Cursor. Cursor documentation[EB/OL]. [2026-10-27]. https://cursor.com/docs.

[38] Cline. Cline: AI coding, open source and open choice[EB/OL]. [2026-10-27]. https://cline.bot.

[39] Langfuse. Observability overview[EB/OL]. [2026-10-27]. https://langfuse.com/docs/observability/overview.

[40] OpenTelemetry. Generative AI semantic conventions[EB/OL]. [2026-10-27]. https://opentelemetry.io/docs/specs/semconv/gen-ai/.

[41] Tree-sitter. Tree-sitter documentation[EB/OL]. [2026-10-27]. https://tree-sitter.github.io/tree-sitter/.

[42] BADERDINOV I, GOLUBEV A, NEKRASHEVICH M, et al. SWE-rebench: an automated pipeline for task collection and decontaminated evaluation of software engineering agents[EB/OL]. (2025-05-26)[2026-10-27]. https://arxiv.org/abs/2505.20411.

[43] BROOKE J. SUS: a quick and dirty usability scale[M]//JORDAN P W, THOMAS B, WEERDMEESTER B A, et al. Usability Evaluation in Industry. London: Taylor & Francis, 1996: 189-194.

[44] HART S G, STAVELAND L E. Development of NASA-TLX (Task Load Index): results of empirical and theoretical research[J]. Advances in Psychology, 1988, 52: 139-183. DOI: 10.1016/S0166-4115(08)62386-9.

四、审核意见

指导教师意见：（课题难度是否适中、工作量是否饱满、进度安排是否合理、工作条件是否具备、是否同意开题等）

指导教师签名：_____ _____年____月____日

专业审核意见：（适当对各项内容给与意见和评价，字数不限。）

负责人签名：_____ _____年____月____日